

Network Security Project — Parts 1 & 2: DNS Cache Poisoning + Attack Analysis

Adam Aberbach

- [Part 1 — DNS Cache Poisoning](#)
 - [1. Lab Setup](#)
 - [2. Resolver Configuration](#)
 - [3. Threat Model](#)
 - [4. Attack Methodology](#)
 - [5. Execution & Evidence](#)
 - [6. Caveats](#)
- [Part 2 — Attack Analysis](#)
 - [7. Wireshark capture and aggregate counts](#)
 - [8. Transaction ID and source port analysis](#)
 - [9. Why the attack succeeded — defenses bypassed](#)
 - [10. Summary](#)

Part 1 — DNS Cache Poisoning

Status: sections 1–4 written from design; sections 5–6 will be filled after live capture (Day 3).

1. Lab Setup

Three Ubuntu 22.04 VirtualBox VMs on a host-only network `192.168.56.0/24`, provisioned by a single Vagrantfile. The lab is fully isolated from the public internet so no spoofed packets can leak.

VM	Hostname	IP	RAM	Role
1	attacker	192.168.56.10	2 GB	Runs <code>spoofer.py</code> (Scapy) and Nginx serving the “PWNED” page on :80

VM	Hostname	IP	RAM	Role
2	resolver	192.168.56.20	2 GB	Runs Unbound (deliberately weakened). Listens on :53
3	victim	192.168.56.30	2 GB	/etc/resolv.conf pinned to 192.168.56.20. Runs dig, curl

Topology (each VM's `ip -4 addr show` output is captured in `docs/screenshots/03-07-full-demo-run.txt`):

```
attacker  enp0s8  192.168.56.10/24  on lab_intnet
resolver  enp0s8  192.168.56.20/24  on lab_intnet
victim    enp0s8  192.168.56.30/24  on lab_intnet
(all VMs also have enp0s3 NAT for vagrant ssh)
```

The Vagrantfile pins the base box to `ubuntu/jammy64` version `20240319.0.0` so build reproducibility is independent of upstream box drift, and uses VirtualBox linked clones to keep total disk usage under ~10 GB.

2. Resolver Configuration

The resolver runs Unbound 1.16+. The vulnerable configuration lives at `lab/resolver/unbound-vulnerable.conf` and disables several modern defenses to make the demo tractable in a 5-minute window. Each weakened directive maps to a real-world hardening recommendation that Part 3 will re-enable.

Directive	Effect	Real-world defense disabled
<code>outgoing-port-permit: "33333"</code> (with <code>outgoing-port-avoid: "0-65535"</code>)	Collapses 16-bit source-port entropy to 1	RFC 5452 source-port randomization
<code>use-caps-for-id: no</code>	No 0x20 mixed-case echo	DNS-0x20 query encoding

Directive	Effect	Real-world defense disabled
<code>harden-glue: no</code>	Accepts unsolicited glue	Bailiwick checking on glue
<code>qname-minimisation: no</code>	Full QNAME sent upstream	RFC 9156 qname minimisation
<code>module-config: "iterator"</code>	DNSSEC validator not loaded	DNSSEC validation (Part 3 fix)
<code>forward-zone target.lab → 192.168.56.99</code>	Legitimate response never arrives	Lab simplification — see §6

In addition, the resolver's kernel has `net.ipv4.conf.all.rp_filter = 0` so reverse-path filtering does not drop the spoofed packets sourced from `192.168.56.99` (an IP not in any local route).

Pre-attack baseline: `dig @192.168.56.20 www.target.lab` from the victim returns SERVFAIL after ~5 s. The same query for `example.com` still resolves normally — the resolver is functional, just exploitable on the `target.lab` zone.

(Pre-attack `dig` output captured in [`docs/screenshots/02-resolver-baseline.txt`])

3. Threat Model

Attacker capability: off-path. The attacker can send arbitrary UDP packets to `192.168.56.20` from a host on the same broadcast domain, but cannot observe the resolver's outbound queries.

Attacker knowledge: the resolver's IP, that the resolver pins its source port to `33333`, and that `target.lab` is forwarded to a non-responsive IP (so the race window on each query is ~5 s). The attacker does not know transaction IDs.

Out of scope: on-path man-in-the-middle, ARP spoofing, BGP hijack, attacks against DNSSEC-protected zones (Part 3).

4. Attack Methodology

`spoofers.py` runs two threads concurrently:

Trigger thread (~6 s cadence): sends an A-query for `www.target.lab` to the resolver. Unbound deduplicates in-flight queries of the same `(qname, qtype)`, so a slow cadence is correct: it ensures each new trigger reliably emits an outbound query rather than being absorbed into a pending one. The cadence matches Unbound's default ~5 s negative-cache TTL.

Flood thread: sweeps every transaction ID 0..65535, sending forged DNS replies that:

- impersonate the forwarder (`IP src = 192.168.56.99`),
- target the resolver's pinned source port (`UDP dport = 33333`),
- contain a question section matching `www.target.lab. A`, and
- contain an answer section binding `www.target.lab. → 192.168.56.10` with a 24-hour TTL.

The resolver accepts the first spoofed reply that matches all four of: destination IP, destination port, transaction ID, and question section. Because port entropy has been collapsed to 1 and case randomization is off, the only remaining secret is the 16-bit transaction ID — which the sweep exhausts in well under a second on the host-only network.

5. Execution & Evidence

The lab was provisioned with `vagrant up` from `lab/vagrant/Vagrantfile`, producing three Ubuntu 22.04 VMs on internal network `lab_intnet`. The end-to-end demo `lab/scripts/run-part1.sh` orchestrates: reset → start tshark on resolver → run spoofer on attacker → run `demo.sh` on victim → copy pcap back. Final result of the recorded run:

```
[+] poisoned in 4.2 seconds (target=www.target.lab → 192.168.56.10)
[+] PASS – www.target.lab resolves to 192.168.56.10 (poisoned)
[+] Part 1 demo SUCCESS (pcap → docs/captures/poison.pcap)
```

(Full demo log: [`docs/screenshots/03-07-full-demo-run.txt`].)

5.1 Pre-attack baseline

[`docs/screenshots/02-resolver-baseline.txt`]

```
$ dig @192.168.56.20 www.target.lab
;; communications error to 192.168.56.20#53: timed out
;; no servers could be reached

$ dig @192.168.56.20 example.com +short
172.66.147.243
104.20.23.154
```

The resolver returns SERVFAIL for `www.target.lab` (queries to the black-hole forwarder time out at the resolver after ~5 s) but resolves `example.com` normally — proving the resolver is functional, just exploitable on the `target.lab` zone.

5.2 Post-attack proof

[`docs/screenshots/04-dig-poisoned.txt`]

```
$ dig @192.168.56.20 www.target.lab
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 51825
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; ANSWER SECTION:
www.target.lab.      86360      IN      A      192.168.56.10
;; Query time: 0 msec
```

Note the TTL `86360` (started at 86400 in the spoofed reply, decreased ~40 s by query time) and `Query time: 0 msec` — the answer is served from the resolver's cache.

[`docs/screenshots/05-curl-pwned.txt`]

```
$ POISONED_IP=$(dig @192.168.56.20 www.target.lab +short)
$ curl --resolve www.target.lab:80:$POISONED_IP http://www.target.lab/
<!doctype html>
<html lang="en">
  <title>PWNED</title>
  ...
  <h1>YOU'VE BEEN PWNED</h1>
  <p>This page was served from 192.168.56.10 because the DNS cache for
    www.target.lab on resolver 192.168.56.20 was poisoned.</p>
```

The HTTP request lands on the attacker's Nginx (which has been hosting the page the entire time) instead of any legitimate server.

5.3 Packet-level evidence (`docs/captures/poison.pcap`)

The pcap was captured on the resolver's `enp0s8` interface for the duration of `run-part1.sh`. Aggregate counts via `tshark`:

Packet class	Count
Total DNS packets	23 843
Spoofed replies (<code>src=192.168.56.99</code> , <code>dport=33333</code>)	23 828
Resolver outbound queries (<code>dst=192.168.56.99</code>)	5
Trigger queries from attacker (<code>192.168.56.10</code> → <code>.20</code>)	3

Interpretation: the spoofer sent 23 828 forged replies sweeping txids sequentially; the resolver fired 5 outbound queries (3 explicit triggers from the spoofer plus 2 retries from the same trigger window) and one of the spoofed replies matched (dst port 33333, correct txid, matching question section, source IP 192.168.56.99) before unbound's forwarder timeout.

[docs/screenshots/06-wireshark-overview.txt] excerpt — first 9 spoofs (note sequential txids 0x0000...0x0008, all targeting dport 33333):

frame	time(s)	src	sport	dst	dport	txid	resp	qname
1	0.000000	192.168.56.99	53	192.168.56.20	33333	0x0000	1	www.target.lab
2	0.000001	192.168.56.99	53	192.168.56.20	33333	0x0001	1	www.target.lab
3	0.000001	192.168.56.99	53	192.168.56.20	33333	0x0002	1	www.target.lab
...								

[docs/screenshots/07-wireshark-spoofed-reply.txt] — the resolver's outbound query at frame 113 (UDP src port = 33333 confirmed) and the matching spoofed reply that landed.

5.4 Reproducibility

Three back-to-back runs with `bash run-part1.sh` in this run produced poison times of 0.4 s, 1.4 s, and 4.2 s respectively. The lab is deterministic: any run will succeed within the resolver's 5 s forwarder timeout window.

6. Caveats

- **Black-hole forwarder.** The resolver forwards `target.lab` to 192.168.56.99 , an unassigned IP. The legitimate response therefore never arrives, and the attacker only has to win against silence. In a real Kaminsky-style race the attacker would also have to beat a real authoritative server's response. The pcap reflects this simplification.
- **Single-record poisoning, no NS-glue.** The spoofed reply only injects one A record. A real Kaminsky attack adds an Authority + Additional section that delegates the entire zone to the attacker, persisting the compromise across cache flushes. We chose the simpler form to fit the 4-day build window.
- **Source-port pinned externally.** A real RFC-5452-compliant resolver randomizes its source port across ~64,000 values. Forcing the port to 33333 reduces the attacker's work factor by $\sim 2^{16}$. The same is true of case randomization (`use-caps-for-id: no`).
- **No DNSSEC.** The validator is unloaded. Part 3 will re-enable it and show that the same attack now fails.

Part 2 — Attack Analysis

7. Wireshark capture and aggregate counts

The pcap `docs/captures/poison.pcap` was captured on the resolver’s `enp0s8` interface for the duration of `lab/scripts/run-part1.sh`. `tshark` aggregates over the entire capture:

Metric	Value
Total packets	23 845
Total DNS packets	23 843
Spoofed replies (<code>192.168.56.99 → 192.168.56.20:33333</code>)	23 828
Resolver outbound queries (<code>192.168.56.20 → 192.168.56.99</code>)	5
Trigger queries from attacker (<code>192.168.56.10 → 192.168.56.20</code>)	3
Successful resolver replies to trigger	1
ICMP port-unreachable from resolver	0

The 0 ICMP-unreachable count is significant: every spoof was either accepted by unbound’s outstanding-query table or silently dropped, never rejected at the kernel level. This confirms `rp_filter=0` is in effect — the kernel did not drop spoofs that claimed source `192.168.56.99` on the wrong route.

8. Transaction ID and source port analysis

8.1 Resolver source port (the entropy that was supposed to protect)

Every one of the resolver’s 5 outbound queries used UDP source port `33333`:

```
$ tshark -Y 'ip.src==192.168.56.20 && ip.dst==192.168.56.99' \
-T fields -e udp.srcport | sort | uniq -c
5 33333
```

A modern RFC 5452-compliant resolver would have spread these 5 queries across roughly 60 000 ephemeral ports (Linux’s default ephemeral range is `32768–60999`, ~28 000 ports; some

implementations use the full 1024–65535 ~64 000 ports). The lab’s pinned port collapses this entropy from ~16 bits to **0 bits**.

8.2 Trigger windows (resolver outbound queries)

Each outbound query opens a ~5-second race window during which unbound will accept a reply matching all four of (dst IP, dst port, qname, txid) . The first three are known to the attacker; only the txid is secret. Five trigger windows in this run:

#	Frame	t (s)	UDP src port	DNS txid
1	113	0.047	33333	0xa6bb (42683)
2	2 267	0.427	33333	0xab44 (43844)
3	4 414	0.808	33333	0x762c (30252)
4	8 812	1.563	33333	0x0c87 (3207)
5	13 030	2.319	33333	0x492c (18732)

Each txid is freshly random, as intended by RFC 5452 — but the txid alone is only 16 bits, and the spoofer can sweep all 65 536 values in a few seconds (see §8.3).

8.3 Spoof flood characteristics

spoofer.py builds the spoofed reply once (UDP checksum disabled per RFC 768) and rewrites only the 2 txid bytes per send through a persistent IPPROTO_RAW socket. Per-second rate from the pcap:

Capture second	Spoofed packets sent
0–1 s	5 609
1–2 s	5 628
2–3 s	5 810
3–4 s	5 860
4–5 s	921 (spoofer detected poison; flood thread stopping)

Sustained rate is ~5 800 packets/sec. One full sweep of the 16-bit txid space takes 65 536 / 5 800 ≈ 11.3 seconds — but the spoofer doesn’t need to complete a full sweep before the trigger fires; it just needs to *pass through* the resolver’s chosen txid value while the trigger window is open.

8.4 Why the attack succeeded — race window timing

Mapping the resolver's chosen txid to the spoofer's progress at the trigger time produces a clear "miss / miss / miss / miss / hit" pattern:

Trigger	t (s)	Resolver txid	Flood at t (txid swept up to)	Trigger window expires	When flood reaches that txid	Match?
1	0.047	0xa6bb (42 683)	~270	5.05 s	~7.4 s	✗ outside window
2	0.427	0xab44 (43 844)	~2 500	5.43 s	~7.6 s	✗ outside window
3	0.808	0x762c (30 252)	~4 700	5.81 s	~5.2 s	✗ ~0.6 s late
4	1.563	0x0c87 (3 207)	~9 100	6.56 s	already passed	✗ flood was at 9 100, must wait next sweep at ~14.6 s
5	2.319	0x492c (18 732)	~13 500	7.32 s	~3.2 s	✓ matched

The winning spoof is **frame 18 742 at t = 3.297 s**, txid = 0x492c. It arrived 0.978 s after trigger #5 fired — well within the ~5 s forwarder timeout — and the flood was passing through this txid at ~3.2 s into its sweep.

The full set of spoofs that match a trigger txid (whether their timing fell inside the window or not):

Frame	Time(s)	src	dst	txid	Within window?
3214	0.601	192.168.56.99	192.168.56.20	0x0c87	no (trigger 4 fired at 1.563 s)
18742	3.297	192.168.56.99	192.168.56.20	0x492c	YES (trigger 5 fired at 2.319 s) ← MATCH

(Frame 3214 carried the right txid for what would *become* trigger 4's query, but it arrived 0.96 s before trigger 4 fired — the resolver's socket on port 33333 wasn't yet bound for this query, so the kernel silently dropped the reply.)

9. Why the attack succeeded — defenses bypassed

Field unbound checks on a reply	Entropy in default config	Entropy in this lab	How the attacker handled it
Destination IP (= resolver's IP)	0 bits	0 bits	Known target
Destination UDP port (resolver's source port for this query)	~16 bits	0 bits	Pinned to 33333 by the lab config
Question section (qname + qtype)	0 bits + 0x20 hash bits	0 bits	Known target name; 0x20 disabled
DNS Transaction ID	16 bits	16 bits	Brute-forced via flood at 5 800 pps
Total entropy attacker must defeat	≥ 32 bits	16 bits	One full sweep ≈ 11 s; matched in 0.978 s

In a hardened resolver the attacker would face combined ≥ 32 -bit entropy (txid $\times$ source port), making a successful match in seconds statistically improbable. By collapsing source-port entropy to zero, the lab reduces the puzzle to a 16-bit brute force — which a single laptop can clear within one forwarder-timeout window with margin to spare.

A secondary factor: the kernel's `rp_filter=0` setting allows packets sourced from `192.168.56.99` to reach unbound at all. With strict reverse-path filtering the resolver would drop the spoofs at the kernel before unbound saw them. (`rp_filter` is a perimeter defense, not a DNS-specific one, but it stops naive off-path source spoofing.)

10. Summary

The cache poisoning attack succeeded in ~3.3 s of pcap time (~4.2 s of wall time after `verify_poisoned`'s 2-second poll caught the result) because the resolver's outbound query had only 16 bits of secret entropy (the DNS transaction ID), and the attacker's flood — sustained at ~5 800 packets/sec — could exhaust those 16 bits within a single ~5-second forwarder-timeout window. The pcap evidence is unambiguous: five trigger windows opened, four passed without the flood landing the right txid in time, and the fifth (txid `0x492c`) matched.

The two structural weaknesses that made this possible — pinned source port and disabled DNSSEC validation — both have well-known defenses. RFC 5452 source-port randomization adds ~16 bits of entropy that would multiply the attacker's required throughput by ~65 000 $\times$; DNSSEC adds a cryptographic check that no amount of brute-force at the resolver level can defeat. Part 3

deploys DNSSEC and demonstrates that the same attack now fails even with the source-port weakness still in place.